

Paterni ponašanja u sistemu za upravljanje klinikom

Patern	Gdje se primjenjuje	Razlog uvođenja
State	Klasa Termin / StatusTermina	Kontrola dozvoljenih prijelaza između stanja termina
Observer	Promjena statusa termina	Automatsko obavješćavanje zainteresovanih komponenti
Strategy	SistemZaSkeniranjQRKoda	Zamjenjivi algoritmi validacije QR koda
Command	zakaziTermin, otkaziTermin	Historijat akcija i podrška za poništavanje operacija
Chain of Responsibility	provjeriDostupnost	Strukturirani lanac provjera dostupnosti termina
Iterator	Iteriranje kroz kolekcije termina, pacijenata i doktora	Uniforman pristup elementima kolekcije bez izlaganja interne strukture
Mediator	Koordinacija servisa pri zakazivanju/otkazivanju termina	Smanjenje direktnih zavisnosti između komponenti sistema
Memento	Čuvanje stanja termina prije izmjene ili otkazivanja	Mogućnost vraćanja objekta na prethodno stanje bez narušavanja enkapsulacije
Template Method	Generisanje izvještaja / tok obrade termina	Definisanje kostura algoritma u baznoj klasi uz dopuštanje potklasama da prepišu određene korake

1 State patern – upravljanje statusom termina

State patern omogućava objektu da mijenja svoje ponašanje u zavisnosti od trenutnog stanja. U kontekstu ovog sistema, klasa Termin prolazi kroz definisani životni ciklus stanja.

Prijedlog implementacije podrazumijeva uvođenje apstraktne klase IStatusTermina sa metodama kao što su zakaziTermin(), otkaziTermin() i evidentiraDolazak(). Svaki konkretan status (ZakazanStatus, OtkazanStatus, RealizovanStatus, PacijentPrisutanStatus) implementirao bi ovu klasu i definisao koja su stanja dozvoljena iz tog stanja.

Na primjer, iz stanja 'zakazan' moguće je prijeći u 'otkazan' ili 'realizovan', ali ne direktno u 'pacijentPrisutan'. Ovaj patern osigurava da logika tranzicija bude centralizovana i konzistentna.

2 Observer patern – obavješćavanje o promjeni stanja

Observer patern definiše zavisnost jedan-prema-više između objekata, tako da kada jedan objekat promijeni stanje, svi zavisni objekti automatski budu obavješćeni. U ovom sistemu, klasa Termin bila bi 'subjekt' (publisher), a različiti dijelovi sistema bili bi 'posmatrači' (subscribers).

Konkretni posmatrači mogli bi biti: NotifikacioniServis (slanje potvrde pacijentu), DnevnikAktivnosti (bilježenje promjene), i RasporedServis (oslobađanje termina pri otkazivanju). Uvođenjem ovog patterna izbjeгла bi se direktna čvrsta sprega između klase Termin i ostalih servisa.

3 Strategy patern – validacija QR koda

Strategy patern omogućava definisanje familije algoritama, enkapsulaciju svakog od njih i njihovu međusobnu zamjenjivost. Klasa SistemZaSkeniranjQRKoda trenutno implementira jednu konkretnu logiku validacije.

Uvođenjem ovog patterna, definisao bi se interfejs IStrategijaValidacije sa metodom validiraj(vrijednostKoda). Konkretno implementacije bile bi npr. OnlineValidacija, OfflineValidacija, ili ValidacijaSaSticanjem. Klasa sistema za skeniranje primala bi željenu strategiju kroz konstruktor ili setter, što bi omogućilo promjenu načina validacije bez izmjene same klase.

4 Command patern – operacije nad terminima

Command patern enkapsulira zahtjev kao objekat, što omogućava pohranjivanje, redanje i poništavanje operacija. Metode zakaziTermin, otkaziTermin i izmijeniTermin u TerminController-u pogodne su kandidate za ovaj patern.

Svaka operacija bila bi predstavljena kao zaseban objekat komande koji implementira interfejs IKomanda sa metodama izvedi() i ponistiIzvršavanje(). Upravitelj komandama (CommandManager) čuvao bi historijat izvršenih komandi, što otvara mogućnost audit loga i opcionalnog mehanizma poništavanja.

5 Chain of Responsibility – provjera dostupnosti

Chain of Responsibility patern prosleđuje zahtjev kroz lanac handlera sve dok jedan od njih ne obradi zahtjev. Metoda provjeriDostupnost u Raspored klasi vrši niz provjera koje su trenutno implementirane kao sekvencijalni if-else blokovi.

Uvođenjem ovog patterna, svaka provjera bila bi zaseban handler: ProvjeraRasporedaDoktora, ProvjeraDostupnostiTermina, ProvjeraKapaciteta, itd. Handleri bi bili poredani u lanac i svaki bi ili obradio zahtjev ili ga proslijedio sljedećem. Ovo olakšava dodavanje novih provjera bez izmjene postojeće logike.

6 Iterator patern

Iterator patern pruža uniforman način pristupa elementima kolekcije bez izlaganja njene interne strukture. U sistemu za upravljanje klinikom, česta je potreba za iteriranjem kroz liste termina, pacijenata ili doktora na različite načine – npr. filtriranje termina po datumu, pregled svih pacijenata određenog doktora, ili prolazak kroz raspored po vremenskom redoslijedu.

Uvođenjem ovog patterna, definisao bi se interfejs IIterator sa metodama sljedeći(), imaSljedećeg() i trenutni(). Konkretno implementacije poput TerminIterator ili PacijentIterator implementirale bi ovaj interfejs za odgovarajuće kolekcije. Na taj način, klijentski kod prolazi kroz kolekcije na jedinstven način bez obzira na to kako su podaci interno organizovani, što olakšava zamjenu interne strukture podataka bez uticaja na ostatak sistema.

7 Mediator patern

Mediator patern definiše objekat koji enkapsulira način na koji skup objekata međusobno komunicira, smanjujući direktne zavisnosti između njih. U sistemu klinike, prilikom zakazivanja ili otkazivanja termina više servisa mora biti koordinirano: RasporedServis, NotifikacioniServis, NaplatniModul i DnevnikAktivnosti.

Bez ovog patterna, svaka komponenta morala bi direktno pozivati ostale, što stvara gustu mrežu zavisnosti. Uvođenjem klase TerminMediator, sve komponente komuniciraju isključivo kroz mediatora koji zna koje akcije treba pokrenuti i kojim redoslijedom. Na primjer, kada dođe do otkazivanja termina, TerminMediator interno poziva otkazivanje u rasporedu, pokretanje refundacije u naplatnom modulu i slanje obavještenja pacijentu. Ovo značajno smanjuje sprezanje između komponenti i olakšava izmjene u toku koordinacije bez diranja pojedinačnih servisa.

8 Memento patern

Memento patern omogućava snimanje i eksternalizaciju internog stanja objekta tako da se objekat kasnije može vratiti u to stanje, bez narušavanja enkapsulacije. U sistemu klinike, korisno je imati mogućnost poništavanja izmjena na terminu – npr. ako administrator greškom promijeni termin, sistem treba biti u stanju vratiti prethodno stanje.

Uvođenjem ovog patterna, klasa Termin (originator) bila bi odgovorna za kreiranje Memento objekta koji sadrži snapshot njenog trenutnog stanja (datum, vrijeme, doktor, status). Klasa HistorijaTermina (caretaker) čuvala bi listu Memento objekata bez uvida u njihov sadržaj. Kada je potrebno poništiti izmjenu, originator se restaurira iz odgovarajućeg Memento objekta. Ovaj patern dobro se nadopunjuje sa Command paternom koji je već uveden u sistemu, jer zajedno pružaju potpunu podršku za audit log i mehanizam poništavanja operacija.

9 Template Method patern – tok obrade termina i generisanje izvještaja

Template Method patern definiše kostur algoritma u baznoj klasi, ostavljajući potklasama da prepisu određene korake bez mijenjanja strukture algoritma. U sistemu klinike, mnogi procesi dijele isti opšti tok ali se razlikuju u pojedinim koracima: generisanje različitih tipova izvještaja (dnevni, sedmični, mjesečni) ili obrada termina za različite medicinske specijalnosti.

Uvođenjem ovog patterna, definisala bi se apstraktna klasa Osnovnilzvještaj sa metodom generiši() koja poziva redom: pripremiPodatke(), formatirajZaglavljje(), popuniSadržaj() i dodajZaključak(). Bazna klasa implementira zajedničke korake, dok apstraktne metode (poput popuniSadržaj()) prepušta konkretnim potklasama: Dnevnilzvještaj, Sedmičnilzvještaj, Mjesečnilzvještaj. Na ovaj način osigurava se doslednost ukupnog toka generisanja uz fleksibilnost u pojedinim koracima, a eliminišu se duplikati koda koji bi nastali kada bi svaka vrsta izvještaja implementirala cijeli algoritam od početka.